C++ Functions Lab Practice

Lab 7— Functions: Parameters & Return Values

Parameters + Return | Parameters + No Return | Logic Building

Restriction: Use only int, float, double, char, bool, arrays. No strings or pointers.

Objectives

1. Pass simple variables/arrays as reference / value parameters to functions.
2. Write functions that receive parameters and return a computed result.
3. Write void functions that receive parameters and perform actions (print, calculate).
4. Build multi-step logic: conditions, loops, and arithmetic inside function bodies.
5. Write function prototypes when definitions appear after main().
6. Call functions correctly from main() using the right arguments.

Part 1: Arrays & Reference Parameters

These questions practice passing arrays to functions and returning multiple values using reference parameters (pass-by-reference with &).

Q1. Define `getMinMax(int arr[], int size, int& minVal, int& maxVal)` that finds both the minimum and maximum values of an integer array using reference parameters. Call it from `main()` with the array `{4, 7, 1, 9, 3, 5, 2, 8}`. (Code provided as reference.)

Expected Output:

```
Minimum value: 1
Maximum value: 9
```

Q2. Define `computeStats(double arr[], int size, double& total, double& average)` that calculates the sum and average of a double array, returning both via reference parameters. Call it from `main()` with `{5.5, 3.2, 8.1, 6.4, 2.8}`. (Code provided as reference.)

Expected Output:

```
Total: 26
Average: 5.2
```

Hint: Initialize total = 0.0. Loop through the array adding each element to total. After the loop, set average = total / size. Both total and average are reference parameters, so main() sees the results directly.

Q3. Define `reverseArray(int arr[], int size)` that reverses the elements of an integer array. Also define `printArray(int arr[], int size)` to display it. Call both from `main()` with `{10, 20, 30, 40, 50}`.

Note: Arrays are by default act as a reference parameters, means any change in the array will be visible in main, so no returning of array is required.

Expected Output:

```
Original: 10 20 30 40 50
Reversed: 50 40 30 20 10
```

Hint: Use two index variables — left = 0 and right = size - 1. Swap arr[left] with arr[right] using a temp variable, then move left++ and right-- until left >= right. Arrays are passed by pointer so the original array is modified directly.

Q4. Define `countVowelConsonant(char arr[], int size, int& vowCount, int& conCount)` that counts how many elements in the array are vowel and how many are consonat, returning both counts via reference parameters. Call it from `main()` with the text “This is a vowel consonant text”.

Expected Output:

```
Vowel count: 9
Consonant count: 16
```

Part 2: Logic Building

These questions combine **void functions with parameters** and more complex logic: multiple conditions, nested loops, and formatted multi-line output.

Q1. Define printFibonacci(int terms) that prints the first 'terms' numbers of the Fibonacci sequence. Call it from main() with terms = 8. (Code provided as reference.)

Expected Output:

```
Fibonacci (8 terms): 0 1 1 2 3 5 8 13
```

Hint: Start with a = 0, b = 1. Each step: print a, then compute next = a + b, shift a = b, b = next.

C++ Code:

```
void printFibonacci(int terms) {
    cout << "Fibonacci (" << terms << " terms): ";
    int a = 0, b = 1;
    for (int i = 0; i < terms; i++) {
        cout << a << " ";
        int next = a + b;
        a = b;
        b = next;
    }
    cout << endl;
}

int main() {
    printFibonacci(8);
    return 0;
}
```

Q2. Define printFactorsOf(int n) that prints all factors of n using a loop. Call it from main() with n = 36. (Code provided as reference.)

Expected Output:

```
Factors of 36: 1 2 3 4 6 9 12 18 36
```

Q3. Define printPrimesUpTo(int limit) that prints all prime numbers from 2 up to limit. Call it from main() with limit = 30.

Expected Output:

```
Primes up to 30: 2 3 5 7 11 13 17 19 23 29
```

Hint: For each number n from 2 to limit, check if it is prime using a nested loop (similar to isPrime() in Part 3).

Q4. Define printMultiplicationGrid(int size) that prints a multiplication grid of size x size. Call it from main() with size = 4.

Expected Output:

```
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
```

Hint: Use nested for loops. Outer loop = row (i), inner loop = column (j). Print i * j with a fixed width.

Q5. Define printDigitFrequency(int n) that counts and prints how many times each digit 0–9 appears in the integer n. Call it from main() with n = 122334455.

Expected Output:

```
Digit frequencies in 122334455:
Digit 1 -> 1 time(s)
Digit 2 -> 2 time(s)
Digit 3 -> 2 time(s)
Digit 4 -> 2 time(s)
Digit 5 -> 2 time(s)
(digits with 0 occurrences are not printed)
```

Hint: Declare int count[10] = {0}. Wait — no arrays! Instead, use 10 separate int variables or a loop with an if-chain to count each digit 0–9 by repeatedly extracting n % 10 and matching.

Q6. Define printCollatz(int n) that prints the Collatz sequence starting from n until it reaches 1, and also prints how many steps it took. The rule: if n is even, next = n/2; if n is odd, next = 3*n + 1. Call it from main() with n = 6 and n = 27.

Expected Output:

```
Collatz(6):  6 3 10 5 16 8 4 2 1   Steps: 8
Collatz(27): 27 82 41 124 ... 1     Steps: 111
```

Hint: Use a while loop. Condition: n != 1. Inside: print n, then apply the rule (use % 2 to check even/odd), increment a counter. After loop print 1 and the step count.

— Excellent Work! —